Matrixmultiplikation von Strassen

Stephan Epp
hjstephan86@gmail.com

14. Juli 2025

Inhaltsverzeichnis

1 Laufzeitanalyse **2**

2 Algorithmus **3**

3 Experimentelle Ergebnisse **4**

4 Optimierung **5**
4.1 Komplexitätsklassen 8
4.2 Erwartete Ergebnisse 9

5 Signalübertragung **10**

6 Der Algorithmus MM-APSP **12**
6.1 Die Rolle des Witness 12
6.2 Randomisierter Ansatz für allgemeine Zeugen 12
6.3 Wiederholung des Experiments 13
6.4 Nachfolger für kürzeste Pfade 14
6.5 Beispiel 15

7 Zusammenfassung **16**

8 Ausblick **17**

Der Algorithmus von Strassen multipliziert zwei $n \times n$ Matrizen A und B effizient, da er nicht 8 sondern nur 7 Matrixmultiplikationen benötigt. In jedem rekursiven Aufruf teilt er A und B und ermittelt das Produkt $C = A \cdot B$ mit einer Laufzeit von $O(n^{2.8074})$.

1 Laufzeitanalyse

Für die Multiplikation von zwei $n \times n$ Matrizen lautet die Rekurrenzgleichung für den Algorithmus von Strassen

$$T(n) = 7\,T(\tfrac{n}{2}) + c\,n^2.$$

Der Algorithmus halbiert in jedem rekursiven Aufruf die beiden $n \times n$ Matrizen zu vier $\frac{n}{2} \times \frac{n}{2}$ Matrizen. Substituieren wir n durch $\frac{n}{2}$, erhalten wir für

$$T(\tfrac{n}{2}) = 7\,T(\tfrac{n}{4}) + c(\tfrac{n}{2})^2 = 7\,T(\tfrac{n}{4}) + \tfrac{c}{4}n^2.$$

Nach dem *ersten* rekursiven Aufruf erhalten wir mit $T(\frac{n}{2})$ eingesetzt in $T(n)$ dann

$$\begin{aligned} T(n) &= 7\,(7\,T(\tfrac{n}{4}) + \tfrac{c}{4}\,n^2) + cn^2 \\ &= 7^2\,T(\tfrac{n}{4}) + \tfrac{7}{4}\,cn^2 + cn^2. \end{aligned}$$

Mit dem *zweiten* rekursiven Aufruf werden die vier $\frac{n}{2} \times \frac{n}{2}$ Matrizen wieder halbiert zu acht $\frac{n}{4} \times \frac{n}{4}$ Matrizen. Damit ist

$$T(\tfrac{n}{4}) = 7\,T(\tfrac{n}{8}) + c(\tfrac{n}{4})^2 = 7\,T(\tfrac{n}{8}) + \tfrac{c}{16}n^2.$$

Wird $T(\frac{n}{4})$ eingesetzt in $T(n)$ ergibt sich

$$\begin{aligned} T(n) &= 7^2\,(7\,T(\tfrac{n}{8}) + \tfrac{c}{16}n^2) + \tfrac{7}{4}cn^2 + cn^2 \\ &= 7^3\,T(\tfrac{n}{2^3}) + \tfrac{7^2}{4^2}cn^2 + \tfrac{7}{4}cn^2 + cn^2. \end{aligned}$$

Betrachten wir nun den k-ten rekursiven Aufruf finden wir für

$$T(n) = 7^k\,T\left(\tfrac{n}{2^k}\right) + cn^2 \sum_{i=0}^{k-1} \left(\frac{7}{4}\right)^i.$$

Zur Vereinfachung belassen wir es bei dem k-ten rekursiven Aufruf auch in $T(n)$ bei k und nicht $k+1$. Kleinere Matrizen als 1×1 Matrizen gibt es nicht, daher können die $n \times n$ Matrizen nur k mal halbiert werden. Der größte Wert, den k annehmen kann, ist $k = \log_2 n$. Damit ist

$$\begin{aligned} T(n) &= 7^{\log_2 n}\,T\left(\frac{n}{2^{\log_2 n}}\right) + cn^2 \sum_{i=0}^{\log_2 n - 1} \left(\tfrac{7}{4}\right)^i \\ &= n^{\log_2 7}\,T(1) + cn^2\,\frac{\left(\frac{7}{4}\right)^{\log_2 n} - 1}{\frac{7}{4} - 1} \\ &= O\left(n^{2.8074}\right) + cn^2\left(\tfrac{7}{4}\right)^{\log_2 n} \\ &= O\left(n^{2.8074}\right) + cn^2 \cdot n^{\log_2 \frac{7}{4}} \\ &= O\left(n^{2.8074}\right) + cn^{2.8074} \\ &= O\left(n^{2.8074}\right). \end{aligned}$$

Algorithmus 1 STRASSEN(A, B)

Eingabe: $\langle A, B\rangle$ mit $n \times n$ Matrizen A, B, $n = 2^k$, $k \in \mathbb{N}$
Ausgabe: $\langle C\rangle$ mit Produktmatrix $C = AB$

1: **if** $n = 1$ **then return** $C = AB$
2: $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$
3: $B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$
4: $P_1 = \textsc{Strassen}(A_{11} + A_{22}, B_{11} + B_{22})$
5: $P_2 = \textsc{Strassen}(A_{21} + A_{22}, B_{11})$
6: $P_3 = \textsc{Strassen}(A_{11}, B_{12} - B_{22})$
7: $P_4 = \textsc{Strassen}(A_{22}, B_{21} - B_{11})$
8: $P_5 = \textsc{Strassen}(A_{11} + A_{12}, B_{22})$
9: $P_6 = \textsc{Strassen}(A_{21} - A_{11}, B_{11} + B_{12})$
10: $P_7 = \textsc{Strassen}(A_{12} - A_{22}, B_{21} + B_{22})$
11: $C_{11} = P_1 + P_4 - P_5 + P_7$
12: $C_{12} = P_3 + P_5$
13: $C_{21} = P_2 + P_4$
14: $C_{22} = P_1 - P_2 + P_3 + P_6$
15: **return** $C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$

2 Algorithmus

Es folgt der Algorithmus 1 von STRASSEN als Pseudocode. Als Eingabe erhalten wir zwei $n \times n$ Matrizen. Der Einfachheit halber wird angenommen, dass n eine Zweierpotenz ist. Zu Beginn prüfen wir, ob die Größe der Matrizen bereits den Wert 1 hat. Haben die Matrizen den Wert 1, geben wir das Produkt AB zurück. In Zeile 2 und 3 werden die Matrizen A und B so definiert, dass in den Zeilen 4 bis 10 die 5 Matrixmultiplikationen jeweils durchgeführt werden. In den Zeilen 11 bis 14 werden 4 Matrizen C_{ij} durch Addition und Subtraktion der Matrizen A_{ij} berechnet und in Zeile 15 Matrix C als Ergebnis zurückgegeben.

Als Idee zum Beweis der Korrektheit betrachten wir das Produkt $A \cdot B$ der zwei Matrizen A und B mit

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \quad \text{und} \quad B = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Zur Vereinfachung beinhalten die Matrizen nur skalare Werte. Das Ergebnis $C = A \cdot B$ ist

$$C = \begin{pmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{pmatrix}.$$

Der Algorithmus von STRASSEN berechnet P_r mit

$$\begin{aligned}
P_1 &= \textsc{Strassen}(a + d, e + h) &&= ae + ah + de + dh, \\
P_2 &= \textsc{Strassen}(c + d, e) &&= ce + de, \\
P_3 &= \textsc{Strassen}(a, f - h) &&= af - ah, \\
P_4 &= \textsc{Strassen}(d, g - e) &&= dg - de, \\
P_5 &= \textsc{Strassen}(a + b, h) &&= ah + bh, \\
P_6 &= \textsc{Strassen}(c - a, e + f) &&= ce + cf - ae - af, \\
P_7 &= \textsc{Strassen}(b - d, g + h) &&= bg + bh - dg - dh.
\end{aligned}$$

Dann werden C_{ij} berechnet mit

$$\begin{aligned}
C_{11} &= P_1 + P_4 - P_5 + P_7 &&= ae + bg,\\
C_{12} &= P_3 + P_5 &&= af + bh,\\
C_{21} &= P_2 + P_4 &&= ce + dg,\\
C_{22} &= P_1 - P_2 + P_3 + P_6 &&= cf + dh,
\end{aligned}$$

wobei z.B. $C_{11} = (ae + ah + de + dh) + (dg - de) - (ah + bh) + (bg + bh - dg - dh) = ae + bg$. Für einen formalen Beweis der Korrektheit verzichten wir auf die vollständige Induktion über $n \in \mathbb{N}$ der $n \times n$ Matrizen.

3 Experimentelle Ergebnisse

Um die theoretische Laufzeitanalyse von Strassens Algorithmus zu überprüfen, wurde eine Python-Implementierung des Algorithmus erstellt und deren Performance mit der einer Standard-Matrixmultiplikation verglichen. Die Experimente wurden für Matrizen unterschiedlicher Größe (n) durchgeführt, wobei n von 4 bis 256 in Schritten von 4 variiert wurde. Für jede Matrixgröße wurden 5 Messungen (Trials) durchgeführt und die durchschnittliche Laufzeit ermittelt. Ein interner Schwellwert (threshold) von 32 wurde für Strassens Algo-

Tabelle 1: Vergleich der Matrixmultiplikationen

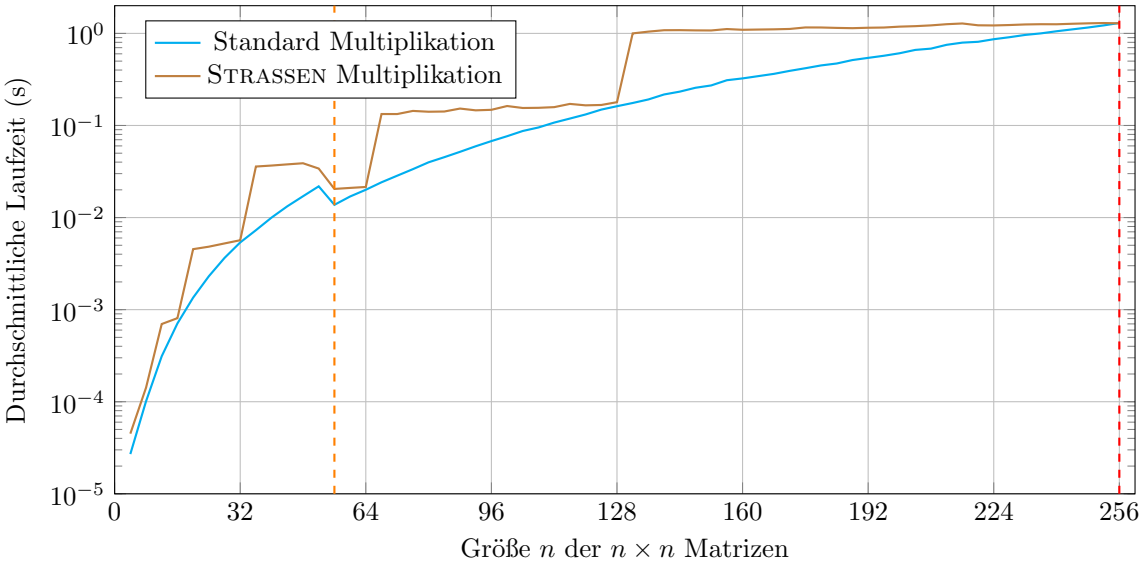
n	Durchschnittliche Laufzeit (s)	
	Standard	Strassen
4	0.000027	0.000045
8	0.000101	0.000142
12	0.000312	0.000698
16	0.000708	0.000808
20	0.001350	0.004538
24	0.002304	0.004827
28	0.003644	0.005236
32	0.005369	0.005674
36	0.007291	0.035827
40	0.010007	0.036722
44	0.013256	0.037815
48	0.017039	0.038888
52	0.021834	0.034057
56	0.013749	0.020460
60	0.016938	0.020990
64	0.020041	0.021484
⋮	⋮	⋮
240	1.055068	1.256394
244	1.105579	1.272495
248	1.158183	1.285980
252	1.221187	1.295652
256	1.298067	1.286444

rithmus festgelegt, was bedeutet, dass für Matrizen, deren Größe kleiner oder gleich 32 ist, auf die Standard-Matrixmultiplikation umgeschaltet wird, um den zusätzliche Aufwand der Rekursion zu reduzieren. Die Systemauslastung vor Beginn der Experimente betrug 0,0% CPU-Auslastung bei 3.38 GB verwendetem RAM von insgesamt 7.01 GB. Nach Abschluss der Experimente betrug die CPU-Auslastung 3.8% und der verwendete RAM 3.43 GB, was

auf eine moderate Systemauslastung während der Messungen hinweist. Die Messergebnisse sind in Tabelle 1 zusammengefasst.

Aus der Tabelle lässt sich ablesen, dass für kleine Matrizen (z.B. $n \leq 64$) die Standard-Matrixmultiplikation tendenziell schneller war oder eine vergleichbare Performance wie Strassens Algorithmus aufwies. Dies ist auf den zusätzlichen Aufwand der Rekursion und der zusätzlichen Matrixadditionen/-subtraktionen bei Strassens Algorithmus zurückzuführen. Insbesondere ist der Sprung in der Laufzeit des Algorithmus bei $n = 36$ (nach dem eingestellten Schwellenwert von $n = 32$) auffällig, was den Wechsel von der optimierten Basis-Multiplikation zur rekursiven Struktur widerspiegelt. Erst ab $n = 256$ übertrifft Strassen die Standardmultiplikation leicht. Die tatsächliche Laufzeit wird jedoch stark vom gewählten Schwellwert und der Effizienz der Implementierung der Basisfälle beeinflusst. Die Abbildung 1 zeigt die Verläufe der benötigten Rechenzeit beider Matrixmultiplikationen. Für $n = 56$ waren sowohl die Standard als auch Strassens Multiplikation schneller als für vorherige $n < 56$.

Abbildung 1: Vergleich der Laufzeit der Matrixmultiplikationen



Strassen verwendet 7 Matrixmultiplikationen für zwei Matrizen der Größe $n \times n$. Lässt sich die Anzahl der benötigten Matrixmultiplikationen reduzieren?

4 Optimierung

Um die Anzahl der benötigten Matrixmultiplikationen von 7 auf 5 zu reduzieren, wird die Diagonale e und h der Matrix B betrachtet. Dadurch wird für n eine Laufzeit im Exponenten von 2.3219 statt 2.807 erreicht. Betrachtet man $P_2 = (c + d) \cdot e = ce + de$ und $P_5 = (a + b) \cdot h = ah + bh$, dann fällt auf, dass P_2 durch P_6 und P_1 ermittelt werden kann,

$$ce = P_6 - cf + ae + af,$$
$$de = P_1 - ae - ah - dh,$$

und P_5 durch P_1 und P_7

$$ah = P_1 - ae - de - dh,$$
$$bh = P_7 - bg + dg + dh.$$

Dazu müssen P_1, P_6 und P_7 berechnet werden bevor P_2 und P_5 ohne Multiplikation bestimmt werden können:

$$P_2 = ce + de = (P_6 - cf + ae + af) + (P_1 - ae - ah - dh) = P_6 + P_1 - cf + af - ah + af,$$
$$P_5 = ah + bh = (P_1 - ae - de - dh) + (P_7 - bg + dg + dh) = P_1 + P_7 - ae - de - bg + dg.$$

Da nur 5 Matrixmultiplikationen verwendet werden, ergibt sich für die Laufzeitanalyse

$$T(n) = 5^{\log_2 n}\, T\Big(\frac{n}{2^{\log_2 n}}\Big) + cn^2 \sum_{i=0}^{\log_2 n - 1} \left(\tfrac{5}{4}\right)^i = n^{\log_2 5}\, T(1) + cn^2 \frac{\left(\frac{5}{4}\right)^{\log_2 n} - 1}{\frac{5}{4} - 1} = O\left(n^{2.3219}\right).$$

Dazu wird der Algorithmus von Strassen zu Strassen-25 so geändert, dass P_2 und P_5 in Zeile 9 und 10 ermittelt werden. Damit P_2 und P_5 ermittelt werden können, müssen die zuvor ermittelten Produkte P_r, $r \in \{1, 6, 7\}$ verwendet werden. Das bedeutet für

$$P_2 = P_6 + P_1 - A_{21}B_{12} + A_{11}B_{12} - A_{11}B_{22} + A_{11}B_{12},$$
$$P_5 = P_1 + P_7 - A_{11}B_{11} - A_{22}B_{11} - A_{21}B_{21} + A_{22}B_{21}.$$

Dabei ist P_2 *abhängig* von P_6 und P_7 und P_5 ist *abhängig* von P_1 und P_7. Damit P_2 und P_5 ohne Matrixmultiplikation ermittelt werden können, müssen die Produkte $A_{ij}B_{ij}$ im vorherigen Rekursionsschritt jeweils gespeichert werden. Damit in jedem Rekursionsschritt

Algorithmus 2 Strassen-25(A, B)

Eingabe: $\langle A, B\rangle$ mit $n \times n$ Matrizen A, B, $n = 2^k$, $k \in \mathbb{N}$
Ausgabe: $\langle C, A_{ij}, B_{ij}\rangle$ mit Produktmatrix $C = AB$ und A_{ij}, B_{ij}

1: **if** $n = 1$ **then return** $C = AB$
2: $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$
3: $B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$
4: $P_1 =$ Strassen-25$(A_{11} + A_{22}, B_{11} + B_{22})$
5: $P_3 =$ Strassen-25$(A_{11}, B_{12} - B_{22})$
6: $P_4 =$ Strassen-25$(A_{22}, B_{21} - B_{11})$
7: $P_6 =$ Strassen-25$(A_{21} - A_{11}, B_{11} + B_{12})$
8: $P_7 =$ Strassen-25$(A_{12} - A_{22}, B_{21} + B_{22})$
9: $P_2 = P_6 + P_1 - A_{21}B_{12} + A_{11}B_{12} - A_{11}B_{22} + A_{11}B_{12}$
10: $P_5 = P_1 + P_7 - A_{11}B_{11} - A_{22}B_{11} - A_{21}B_{21} + A_{22}B_{21}$
11: $C_{11} = P_1 + P_4 - P_5 + P_7$
12: $C_{12} = P_3 + P_5$
13: $C_{21} = P_2 + P_4$
14: $C_{22} = P_1 - P_2 + P_3 + P_6$
15: **return** $C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$

die Produkte $A_{ij}B_{ij}$ effizient gespeichert werden können, muss auf eine Datenstruktur mit einem eindeutigen Schlüssel (k, i_A, j_A, i_B, j_B) in $O(1)$ zugegriffen werden, $n = 2^k$, $k \in \mathbb{N}$.

Satz 4.1. *Es gibt keinen Teile-und-Herrsche Algorithmus zur Multiplikation von $n \times n$ Matrizen, der durch Nutzen von Abhängigkeiten weniger als 8 Matrixmultiplikationen benötigt und damit auch weniger als 5 Matrixmultiplikationen benötigt.*

Beweis. Angenommen, es gibt einen Teile-und-Herrsche Algorithmus zur Multiplikation von $n \times n$ Matrizen, der mit weniger als 5 Matrixmultiplikationen auskommt, dann muss dieser Algorithmus auch Abhängigkeiten $P_r = (P_s, P_t, A_{ij}B_{ij})$ verwenden, mit denen P_r

ermittelt werden kann, $r < 5$. Da $dim(A_{ij}) = dim(B_{ij}) = \frac{n}{2}$, ist es nicht möglich $C = AB$ zu berechnen. Das gesamte Bild der *projezierten* Produktmatrix C kann nur projeziert werden, wenn mindestens mehr als die Hälfte beider Matrizen A_{ij} und B_{ij} zum Projezieren verfügbar ist. □

Satz 4.2. *Das optimale Teile-und-Herrsche Prinzip ist die effizienteste Methode, um Probleme zu lösen, wenn die Probleminstanz bei jedem rekursiven Aufruf halbiert wird.*

Beweis. In diesen drei Kriterien ist der Beweis zu führen: (1) Rekursionstiefe, (2) Anzahl der Teilprobleme, (3) Größe eines Teilproblems. Dabei wird schnell klar, dass es alles in der Halbierung zusammengefasst optimal ist. Denn die Halbierung ist nichts anderes als eine Verdoppelung im Nenner.

Zu (1): Wichtig in der Analyse der Laufzeit eines Algorithmus oder einer Lösungsvorschrift, bestehend aus einfachen Operationen und dem Aufruf von Funktionen, ist das Betrachten der gerufenen Funktionen. Weniger wichtig sind einfache Operationen und Zuweisungen. Daher ist für die Laufzeitanalyse die Rekursionstiefe von wichtigster Bedeutung (sieht man z.B. beim Algorithmus von Strassen oder beim Algorithmus Merge-Sort).

Die Rekursionstiefe ist logarithmisch. Kleiner kann die Tiefe für die gesamte Lösung nicht sein, da das Problem mit der 2 im Nenner logarithmisch geteilt wird. Die Funktion, die am stärksten wächst und dabei immer noch eine Umkehrfunktion hat, ist die Exponentialfunktion. Daher ist ihre Umkehrfunktion am schnellsten in der Reduzierung der Rekursionstiefe für die Laufzeitanalyse für Teile-und-Herrsche Algorithmen. Eine Erhöhung des Nenners führt zwangsläufig zu einer Veränderung der anderen beiden Kriterien.

Zu (2), (3): Für die Anzahl der Teilprobleme und für die Größe eines Teilproblems gilt: Angenommen, der Nenner wird vergrößert. Dann wird die ursprüngliche Probleminstanz aber nicht in ihrem leichtesten Punkt geteilt. Denn ein Problem ist nur da am leichtesten zu teilen, wo sein Schwerpunkt liegt. Es gibt für jede Probleminstanz nur einen Schwerpunkt. Wenn der Nenner ganzzahlig verkleinert wird, wird nicht mehr geteilt. Es muss ganzzahlig verkleinert werden, da z.B. ein einzelnes Array-Element nicht teilbar ist. Die Zahl e scheidet damit in der Wahl der Exponentialfunktion aus. Daher muss die Exponentialfunktion 2 als Basis haben. □

Auffällig ist, dass nur das Problem, bei dem zwei Instanzen durch einen Operator verbunden sind, mit dem optimalen Teile-und-Herrsche Prinzip zu lösen ist. Das sind Probleme mit Operatoren wie z.B., $+$, $-$, $<$. Im Prinzip werden diese Probleme an ihrem Schwerpunkt gelöst, wo sie trotz ihrer Schwere am leichtesten zu tragen sind, wie bei einer Wippe mit zwei gleichen Gewichten jeweils am äußersten Ende. Die Wippe ist dann exakt horizontal ausgerichtet im Gleichgewicht.

Lemma 4.1. *Schneller als mit $T(n) = O(n^{2.3219})$ können zwei $n \times n$ Matrizen nicht multipliziert werden.*

Beweis. Der Beweis folgt daraus, dass Strassen-25(A, B) zwei $n \times n$ Matrizen nach dem optimalen Teile-und-Herrsche Prinzip multipliziert. □

Satz 4.3. *Wenn es bei einer Lösung erforderlich ist, jedes Element der Eingabe während der Abarbeitung der Lösungsvorschrift einmal zu betrachten, dann gibt es keine bessere Laufzeit als die logarithmische.*

Beweis. Wenn Laufzeit so aufgefasst wird, dass in jedem Schritt ein Register besucht wird, dann kann die Anzahl der unterschiedlich besuchten Register nicht weniger als logarithmisch sein. Die Sonnenblumenkerne in einer Sonnenblume sind logarithmisch angeordnet und damit optimal untergebracht mit minimal wenig Platz. Sie sind im goldenen Winkel angebracht. Würden sie besser angebracht werden können, dann wäre der goldene Winkel kein goldener Winkel. □

Lemma 4.2. *Das optimale Teile-und-Herrsche Prinzip ist optimal unter allen Lösungsprinzipien, wo es während der Abarbeitung der Lösungsvorschrift erforderlich ist, jedes Element der Eingabe während der Lösung einmal zu betrachten.*

Beweis. Der Beweis folgt daraus, dass Algorithmen, die nach dem optimalen Teile-und-Herrsche Prinzip Lösungsvorschriften abarbeiten, logarithmische Laufzeit haben müssen. □

4.1 Komplexitätsklassen

Algorithmen folgen einem Lösungsprinzip nach dem sie ein Problem lösen. Es gibt zum Beispiel das optimale Teile-und-Herrsche Prinzip oder das Prinzip der dynamischen Programmierung. Es kann sich lohnen, die Klasse **P** in Komplexitätsklassen einzuteilen, bei der eine Klasse durch ein Lösungsprinzip und ihre charakteristische Laufzeit beschrieben wird. Zum Beispiel kann man sich die Klasse vorstellen, in der alle Probleme enthalten sind, die nach dem optimalen Teile-und-Herrsche Prinzip zu lösen sind und nicht besser. Da die charakteristische Laufzeit dieser Klasse die logarithmische ist, könnte sie den Namen **L** haben. Man kann sich auch die Klasse $\mathbf{P_2}$ vorstellen, in der alle Probleme enthalten sind, die nach dem Prinzip der dynamischen Programmierung zu lösen sind und für deren Lösung quadratische Laufzeit benötigt wird.

Definiton 4.1. *Die Klasse* **P** *enthält die Menge aller Probleme, für die es jeweils einen Algorithmus gibt, der das Problem in polynomieller Laufzeit löst.*

Definiton 4.2. *Die Klasse* **NP** *enthält die Menge aller Probleme, von denen bekannt ist, dass es für sie jeweils keinen Algorithmus gibt, der das Problem noch in polynomieller Laufzeit lösen kann.*

Bisher hat man vermutet, dass zwischen **P** und **NP** eine Beziehung derart existiert, dass $\mathbf{P} \subseteq \mathbf{NP}$. Diese Vermutung ist aber nicht richtig.

Satz 4.4. *Die Klassen* **P** *und* **NP** *sind gleich, d.h.,* $\mathbf{P} = \mathbf{NP}$.

Beweis. Betrachte das Problem **S**, die eindeutige Schneeflocke. Gegeben ist eine Schneeflocke mit 5 gleichlangen Armen. Wie wird diese Schneeflocke eindeutig unter allen bereits vorhandenen Schneeflocken? Finde einen Algorithmus, der in polynomieller Zeit immer eine neue Schneeflocke erzeugt. Beweis durch Induktion:
IB: Sei S die Menge aller eindeutigen Schneeflocken, o.B.d.A.
IA: $|S| = 0$ trivial, $|S| = 1$ trivial
IS: $|S| \to |S| + 1$, $|S| = n, n \in \mathbb{N}$
Ziel: Mache aus 8 Knoten wieder 6 Knoten mit gleicher Kantenlänge in polynomieller Zeit. Berechne dazu die Restklasse 6. Jeder Knoten berechnet eine Funktion $f(b_1, \ldots, b_n) = \{0, \ldots, 9\}$, $b_i \in \{0, 1\}$. Bei Hinzufügen zwei neuer Knoten erhöht sich die Wertemenge des vorhandenen Graphen um maximal $18 \equiv 0 \mod 6$.
Die Vermutung liegt nahe, das Problem **S** als NP-vollständig einzuschätzen. Da dieses Problem aber in polynomieller Zeit lösbar ist, ist es echt in **P**. □

Da $\mathbf{P} = \mathbf{NP}$, lohnt es sich gerade auch die Probleme genauer zu betrachten, von denen man vorher vermutete, dass es für sie keinen Algorithmus gibt, der das Problem in polynomieller Laufzeit lösen kann.

Vielleicht kann man in Zukunft ein Problem der Klasse $\mathbf{P_2}$ auch effizienter lösen und damit der Klasse **L** zuordnen. Man kann sich die Klasse $\mathbf{P_3}$ vorstellen, in der alle Probleme enthalten sind, die nach dem Prinzip der dynamischen Programmierung zu lösen sind und für deren Lösung kubische Laufzeit benötigt wird (die Lösungstabelle hat eine dritte Dimension). Auch hier kann es sein, dass für ein Problem dieser Klasse eine effizientere Lösung gefunden wird und sie damit einer leichteren Komplexitätsklasse zugeordnet werden

kann. Da ein Problem auch immer weniger effizient, d.h., langsamer, gelöst werden kann, ist es nicht so, dass z.B. ein Problem der Klasse $\mathbf{L}$ auch der Klasse $\mathbf{P_2}$ zuzuordnen ist, da man Probleme in der Praxis nicht langsamer löst. Alle Klassen sind in $\mathbf{P}$ enthalten aber untereinander nicht:

$$\mathbf{L} \subset \mathbf{P}, \quad \mathbf{P_2} \subset \mathbf{P}, \quad \mathbf{P_3} \subset \mathbf{P}, \quad \mathbf{L} \not\subset \mathbf{P_2}, \quad \mathbf{P_2} \not\subset \mathbf{P_3}, \quad \mathbf{L} \not\subset \mathbf{P_3}.$$

Beispiele für Probleme, die in $\mathbf{L}$ enthalten sind, sind die $n \times n$ Matrixmultiplikation und das Sortieren von ganzen Zahlen. Beide Probleme lassen sich mit dem optimalen Teile-und-Herrsche Prinzip lösen.

4.2 Erwartete Ergebnisse

Bei der Analyse der zu messenden Laufzeit von Algorithmus Strassen-25 ist zu erwarten, dass dieser mit nur 5 Matrixmultiplikationen deutlich weniger Rechenzeit benötigt als der Algorithmus von Strassen mit 7 Matrixmultiplikationen.

Die Tabelle 2 zeigt die theoretische Anzahl benötigter Operationen für die Matrixmultiplikation in Abhängigkeit von n. Die Operationen sind elementare arithmetische Operationen wie Multiplikationen und Additionen/Subtraktionen von Skalaren. Die Anzahl der Operationen in der Tabelle sind asymptotisch und ignorieren konstante Faktoren, um die Skalierungseffekte zu betonen. Die Wahl des geeigneten Algorithmus zur Matrixmultiplikation hängt

Tabelle 2: Vergleich der theoretischen Anzahl an Operationen

n	Standard	Strassen	Strassen-25	Matrixgröße ($n \times n$)
10	$10^3 = 1.000$	$10^{2.8074} \approx 642$	$10^{2.3219} \approx 209$	100
100	$100^3 = 10^6$	$100^{2.8074} \approx 6.4 \times 10^5$	$100^{2.3219} \approx 2.09 \times 10^4$	10^4
1.000	$1.000^3 = 10^9$	$1.000^{2.8074} \approx 6.4 \times 10^8$	$1.000^{2.3219} \approx 2.09 \times 10^7$	10^6
10.000	$10.000^3 = 10^{12}$	$10.000^{2.8074} \approx 6.4 \times 10^{11}$	$10.000^{2.3219} \approx 2.09 \times 10^9$	10^8
100.000	$100.000^3 = 10^{15}$	$100.000^{2.8074} \approx 6.4 \times 10^{14}$	$100.000^{2.3219} \approx 2.09 \times 10^{11}$	10^{10}
1.000.000	$1.000.000^3 = 10^{18}$	$1.000.000^{2.8074} \approx 6.4 \times 10^{17}$	$1.000.000^{2.3219} \approx 2.09 \times 10^{13}$	10^{12}
10^7	10^{21}	6.4×10^{20}	2.09×10^{15}	10^{14}
10^8	10^{24}	6.4×10^{23}	2.09×10^{17}	10^{16}

stark von der Größe der Matrizen und dem spezifischen Anwendungsbereich ab. Interessant ist, dass schon für $n = 100$ nur 10^4 Operationen benötigt werden von Strassen-25 und dagegen 10^5 Operationen von Strassen. Für $n \geq 10.000$ unterscheiden sich Strassen-25 und Strassen um 2 Zehnerpotenzen und für $n \geq 10^8$ unterscheiden sie sich um 6 Zehnerpotenzen. Praktische Anwendungsbereiche für die $n \times n$ Matrixmultiplikation sind die folgenden:

1. **Kleine Matrizen** ($n < 200$): Für kleinere Matrizen, wie sie in der Computergrafik oder bei einfachen linearen Gleichungssystemen vorkommen, ist der Standardalgorithmus mit $O(n^3)$ aufgrund seines geringen zusätzlichen Aufwands für die Rekursion und optimaler Cache-Nutzung die erste Wahl.

2. **Mittlere bis große Matrizen** ($n \approx 200 - 10.000$): Im wissenschaftlichen Rechnen, bei Optimierungsproblemen oder im maschinellen Lernen dominieren weiterhin optimierte Implementierungen des Standardalgorithmus.

3. **Sehr große Matrizen** ($n \geq 10.000$): Bei großen Matrizen, wie sie in Big Data Analysen oder beim Training umfangreicher Deep-Learning-Modelle auftreten, wird der Bedarf an Alternativen zu optimierten Standardalgorithmen deutlich. Damit gewinnt der Algorithmus von Strassen an Attraktivität und Relevanz für die Bewältigung rechenintensiver Multiplikationen.

5 Signalübertragung

Für die Signalübertragung ist es wichtig, das Signal vom Sender zum Empfänger sicher zu übertragen. Welche kleinste Einheit eignet sich dabei als die optimale? Bisher betrachtet wurde: Es gibt die 2 zum Teilen beim optimalen Teile-und-Herrsche Prinzip. Es gibt die Exponentialfunktion zur Basis 2. Es gibt die logarithmische Laufzeit. Es das Binärsystem, bei dem die kleinste Einheit durch 2 Zustände repräsentiert wird.

Satz 5.1. *Das Binärsystem ist optimal für die Signalübertragung, bei der elektronische Signale genutzt werden: für an (1) und für aus (0).*

Beweis. Mit einer 2 Bitfolge können 4 Werte übertragen werden, mit einer 3 Bitfolge können 8 Werte übertragen werden. Die Anzahl der übertragbaren Werte ist exponentiell zur Basis 2 in der Anzahl der Bits in der zu übertragenden Bitfolge. Das heißt, die Anzahl an Bits ist logarithmisch zur Wertigkeit, die übertragbar ist. Warum ist 3 oder 4 als Basis nicht besser? Es ist deshalb nicht besser, da man vom Querschnitt der Leitung aus betrachtet so einen maximalen Abstand zwischen (0) und (1) hat zur sicheren, lesbaren Signalübertragung und Störungen (wie durch Rauschen) so maximal entgegengewirkt wird. In der Mitte des Leitungsquerschnitts ist der Schwerpunkt der Leitung, da hier die Leitfähigkeit maximal bzw. das Rauschen minimal ist. Nur bei 2 Zuständen (0) und (1) mit gleichem Abstand zum Schwerpunkt der Leitung ist der Abstand aller Signale voneinander (maximal) und zum Schwerpunkt der Leitung (minimal) optimal. □

Das heißt, der Leitungsquerschnitt wird somit so klein wie möglich gehalten und damit ist so wenig Leitungsmaterial wie nötig erforderlich. Es ist aus wirtschaftlicher und fairer Sicht nicht vertretbar, städtische Haushalte mit einem anderen als dem Binärsystem für die Signalübertragung zu versorgen. Zugleich verfügen ländliche Gebiete oft nicht über dieselbe Internetbandbreite wie Städte. Die Debatte um die Nutzung von Quantenphysik für die Signalübertragung erübrigt sich somit unter wirtschaftlichen und gerechten Gesichtspunkten. Solange die Quantenphysik nicht für die Signalübertragung eingesetzt wird, ist auch die Verarbeitung von Signalen bzw. Daten mittels Quantenphysik nicht sinnvoll.

Die *Shannon-Hartley-Gleichung* beschreibt die maximale theoretische Datenrate, d.h., *Kanalkapazität C*, die über einen Kommunikationskanal fehlerfrei übertragen werden kann, unter Berücksichtigung von Bandbreite und Rauschen. Sie lautet:

$$\mathrm{C} = \mathrm{B} \cdot \log_2\left(1 + \frac{\mathrm{S}}{\mathrm{N}}\right), \quad \text{dabei ist}$$

C Kanalkapazität in Bit pro Sekunde (bit/s)

B Bandbreite des Kanals in Hertz (Hz)

S Durchschnittliche Signalleistung über den Kanal

N Durchschnittliche Rauschleistung über den Kanal

S/N Signal-Rausch-Verhältnis (SNR)

Das Signal-Rausch-Verhältnis (SNR) ist dabei der Quotient aus der durchschnittlichen Leistung zur Signalübertragung und der durchschnittlichen Rauschleistung im Kanal:

$$\mathrm{S/N} = 10 \cdot \log_{10}\left(\frac{\mathrm{P_{Sign}}}{\mathrm{P_{Nois}}}\right).$$

Beträgt die durchschnittliche Signalleistung zum Beispiel $\mathrm{P_{Sign}} = 100\mathrm{mW}$ und die durchschnittliche Rauschleistung $\mathrm{P_{Sign}} = 1\mathrm{mW}$, dann ist

$$\mathrm{S/N} = 10 \cdot \log_{10}\left(\frac{100\mathrm{mW}}{1\mathrm{mW}}\right) = 20.$$

Das Signal-Rausch-Verhältnis wird nicht in dB angegeben, da es nur ein Verhältnis beschreibt, in dem zwei Leistungen, die Signalleistung und die Rauschleistung, zueinander stehen. Tabelle 3 zeigt typische SNR-Wertebereiche. Bei einem SNR, das größer als 40 ist, ist eine optimale Signalübertragung möglich. Hochpräzise Messsysteme und Spezialanwen-

Tabelle 3: Typische SNR-Wertebereiche und deren Bedeutung für die Signalqualität

SNR	Qualitätseinstufung	Beschreibung / Typische Auswirkungen	Anwendungsbeispiele (Tendenz)
< 0	Unbrauchbar	Rauschleistung überwiegt Signalleistung deutlich. Keine Kommunikation möglich oder extrem gestört.	Sehr schlechte/keine Mobilfunkverbindung, gestörtes WLAN am Rand.
$0 - 5$	Sehr schlecht	Signal kaum von Rauschen zu unterscheiden. Häufige Abbrüche, extrem langsame/unzuverlässige Datenübertragung.	Schwaches WLAN, sehr schlechter Radioempfang.
$5 - 10$	Schlecht	Verbindung instabil, Störungen. Datenraten stark eingeschränkt, viele Wiederholungen nötig.	Grundlegende Internetnutzung (E-Mails), VoIP mit Aussetzern.
$10 - 15$	Akzeptabel	Brauchbare Verbindung. Leichte Qualitätseinbußen, gelegentliche Verzögerungen. Für Grundaufgaben ausreichend.	Web-Browse, normale Videoanrufe, nicht-kritisches Streaming.
$15 - 25$	Gut	Stabile, zuverlässige Verbindung mit guter Qualität. Für die meisten Anwendungen ausreichend.	HD-Streaming, Online-Gaming (Casual), allgemeine Netzwerkaktivitäten.
$25 - 40$	Sehr Gut	Exzellente Bedingungen, kaum Störeinflüsse. Hohe Datenraten und sehr stabile Verbindungen.	4K-Streaming, anspruchsvolles Online-Gaming, professionelle Audio/Video, Unternehmensnetzwerke.
> 40	Optimal	Außergewöhnlich klare, störungsfreie Signale. Ideal für kritische Anwendungen, höchste Signalintegrität.	Hochpräzise Messsysteme, Laborequipment, Spezialanwendungen.

dungen nutzen dieses Verhältnis. Bei einem SNR, das zwischen 25 und 40 liegt, ist die Qualität für die Signalübertragung sehr gut und ermöglicht z.B. 4K-Streaming. Bei einem SNR zwischen 10 und 15 ist die Qualität akzeptabel für normale Videoanrufe oder nicht-kritisches Streaming.

6 Der Algorithmus MM-APSP

Die effiziente Matrixmultiplikation hat einige praktische Anwendungsbereiche, z.B. beim *Finden der kürzesten Wege für alle Knoten* $v \in V$ in einem Graphen $G = (V, E)$. Das Problem, den kürzesten Weg für einen Startknoten $s \in V$ zu finden, löst z.B. die *Tiefensuche* in $O(|V| + |E|)$. Wie effizient kann das Problem *All-Pairs-Shortest-Paths (APSP)* gelöst werden?

Betrachte die beiden Booleschen $n \times n$ Matrizen A und B, deren Einträge entweder 0 oder 1 sind. Das Boolesche Produkt P von A und B ist eine Matrix, deren Einträge p_{ij} wie folgt definiert sind:

$$p_{ij} = \bigvee_{k=1}^{n} (a_{ik} \wedge b_{kj}).$$

Hierbei repräsentieren $\bigvee$ und $\wedge$ die Booleschen Operatoren ODER und UND. Dies bedeutet, dass $p_{ij} = 1$ genau dann ist, wenn es mindestens ein k gibt, für das sowohl $a_{ik} = 1$ als auch $b_{kj} = 1$ gilt.

Das Boolesche Produkt $P = AB$ kann effizient berechnet werden. Man kann die 0/1-Einträge als ganze Zahlen behandeln und ein herkömmliches Matrixprodukt M (mit ganzen Zahlen) berechnen. Dann ist $p_{ij} = 1$ genau dann, wenn der entsprechende Eintrag m_{ij} in der Matrix M größer als 0 ist. Die Zeitkomplexität für diese Berechnung liegt bei $O(M(n))$, wobei $M(n)$ die Zeit ist, die zur Multiplikation zweier $n \times n$ Matrizen benötigt wird.

6.1 Die Rolle des Witness

Der Algorithmus BMM-Witness geht über die reine Feststellung der Erreichbarkeit hinaus. Er liefert zusätzlich einen *Zeugen* für jede gefundene Verbindung. Ein Zeuge für einen Pfad $i \to k \to j$ ist der Zwischenknoten k. Der Algorithmus erstellt eine dritte Matrix, die *Witness-Matrix* W, in der W_{ij} den Index des ersten gefundenen Zwischenknotens k speichert, für den $A_{ik} \wedge B_{kj} = 1$ ist. Wenn kein solcher Pfad existiert ($C_{ij} = 0$), dann ist $W_{ij} = 0$.

Dies ist besonders nützlich, um nicht nur zu wissen, ob ein Pfad existiert, sondern auch wie man dorthin gelangt. Im Kontext von Graphen kann man so die direkten Nachfolger auf einem 2-Schritt-Pfad identifizieren. Der Algorithmus zur Berechnung des Booleschen Matrixprodukts C und der Witness-Matrix W ist relativ einfach und ähnelt der Standard-Matrixmultiplikation, nur mit angepassten Operationen.

Der Algorithmus durchläuft alle möglichen Tripel (i, j, k). Wenn eine Kante von i nach k und eine Kante von k nach j existiert ($A_{ik} = 1$ und $B_{kj} = 1$), dann wird C_{ij} auf 1 gesetzt. Gleichzeitig wird der Index k (hier als $k + 1$ gespeichert, um 0 als Default-Wert für *kein Zeuge* zu reservieren) in W_{ij} eingetragen, sofern noch kein Zeuge für diese Position gefunden wurde. Die Laufzeit dieses Algorithmus ist $O(n \cdot m \cdot p)$, also $O(n^3)$ für quadratische Matrizen wie für die Standard-Matrixmultiplikation.

6.2 Randomisierter Ansatz für allgemeine Zeugen

Eine allgemeine Lösung für das Finden von Zeugen kann mithilfe von Randomisierung erreicht werden. Sei w_{ij} die Anzahl der Zeugen für $p_{ij} = 1$. Wir definieren eine modifizierte Matrix $\hat{A}$ mit $\hat{a}_{ik} = r_k \cdot a_{ik}$, wobei r_k eine Zufallsvariable ist. Genauer gesagt, $r_k = 1$ mit einer Wahrscheinlichkeit π und $r_k = 0$ mit Wahrscheinlichkeit $1 - \pi$. Die Wahrscheinlichkeit π wird so gewählt, dass $1/(2w_{ij}) \leq \pi < 1/w_{ij}$ ist.

Algorithmus 3 BMM-Witness(A, B)

Eingabe: $\langle A, B\rangle$ mit $n \times m$ Matrix A und $m \times p$ Matrix B
Ausgabe: $\langle C, W\rangle$ mit $n \times p$ Produktmatrix C und $n \times p$ Witness-Matrix W

```
1:  Initialisiere C als n × p Matrix mit Nullen
2:  Initialisiere W als n × p Matrix mit Nullen
3:  for i = 1 to n do
4:      for j = 1 to p do
5:          for k = 1 to m do
6:              if A_ik = 1 ∧ B_kj = 1 then
7:                  C_ij = 1
8:                  if W_ij = 0 then            ▷ Speichere den ersten gefundenen Zeugen
9:                      W_ij = k + 1            ▷ Oft k + 1 um 0 als kein Zeuge zu nutzen
10: return ⟨C, W⟩
```

Der Kern des randomisierten Ansatzes ist die Behauptung, dass die Wahrscheinlichkeit, dass die Summe $\sum_{k=1}^{n} \hat{a}_{ik} b_{kj}$ ein Zeuge für $p_{ij} = 1$ ist, mindestens $1/(2e)$ beträgt, wobei e die Eulersche Zahl ist. Dies kann wie folgt bewiesen werden: Angenommen, es gibt w *weiße Kugeln* (Zeugen) und $n - w$ *schwarze Kugeln* (keine Zeugen). Wenn jede der n Kugeln unabhängig voneinander mit Wahrscheinlichkeit π ausgewählt wird, ist die Wahrscheinlichkeit ρ, dass genau eine weiße Kugel gewählt wird, gegeben durch:

$$\rho = w \cdot \pi \cdot (1 - \pi)^{w-1}$$

Unter Verwendung der Grenzen für π erhalten wir für $w > 1$: $\rho > (1/2)(1 - 1/w)^{w-1} \geq 1/(2e)$. Für $w = 1$ gilt $\rho \geq 1/(2e)$ ebenfalls.

6.3 Wiederholung des Experiments

Ein einzelnes Experiment, das mit Wahrscheinlichkeit $\geq 1/(2e)$ einen Zeugen liefert, ist möglicherweise nicht ausreichend. Durch mehrfache Wiederholung des Experiments kann die Wahrscheinlichkeit, genau einen Zeugen zu erhalten, erhöht werden. Bei N Versuchen ist die Wahrscheinlichkeit des Scheiterns in allen Versuchen kleiner als $(1-1/(2e))^N \leq e^{-N/(2e)}$. Wählt man $N = 2ec \log n$, so ist die Fehlerwahrscheinlichkeit kleiner als $1/n^c$ (nachfolgend ist mit log immer $\log_2$ gemeint). Dies bedeutet, dass nur für wenige Einträge (i, j) ein Zeuge nicht gefunden wird.

Da w_{ij} für jedes (i, j) unterschiedlich sein kann und uns nicht bekannt ist, versucht der Algorithmus verschiedene Wahrscheinlichkeiten $\pi_s = 1/2^s$ für $s = 0, \ldots, \lceil \log n \rceil$. Dies wird erreicht, indem man ein Startset $R = \{1, \ldots, n\}$ hat und iterativ ein (1/2)-Sample aus R zieht, wobei jedes Element unabhängig mit Wahrscheinlichkeit 1/2 ausgewählt wird. Im i-ten Iterationsschritt ist k dann mit Wahrscheinlichkeit $1/2^i$ in R. Dass mit wachsendem i die Menge R kleiner wird, ist gleich wichtig in der Laufzeitanalyse und damit für die Effizienz des Algorithmus, d.h., für seine *erwartete Laufzeit*. Der Algorithmus BMM-Witness findet Zeugen für die Boolesche Matrixmultiplikation. Die Schleife in den Zeilen 4 – 10 gewährleistet, dass für jedes Paar (i, j) eine Wahrscheinlichkeit nahe $1/w_{ij}$ ausprobiert wird. Dies geschieht $2ec \log n$ mal, sodass die Wahrscheinlichkeit, einen Zeugen in dieser Schleife nicht zu finden, höchstens $1/n^c$ beträgt. Da es n^2 Paare (i, j) gibt, ist die erwartete Anzahl fehlgeschlagener Paare höchstens $1/n^{c-2}$. Setzt man $c = 1$, ist dies höchstens n, und die erwartete Laufzeit der Zeilen 11-13 (Brute Force) beträgt $O(n^2)$. Das ist beachtlich, da die Brute Force Methode sonst Laufzeit $O(n^3)$ hat. Die dominierende Operation in der Schleife 4 – 10 ist die Matrixmultiplikation in Zeile 6. Daher beträgt die erwartete Gesamtlaufzeit $O(M(n) \log^2 n)$. Die Korrektheit des Algorithmus wird dadurch sichergestellt, dass Zeugen nur dann gesetzt werden, wenn sie direkt überprüft wurden.

Algorithmus 4 BMM-Witness(A, B)

Eingabe: $\langle A, B\rangle$ mit $n \times m$ Matrix A und $m \times p$ Matrix B
Ausgabe: $\langle W\rangle$ mit $n \times p$ Witness-Matrix W

```
W = −AB                                  ▷ Initialisiere W (mit negativen Werten für fehlende Zeugen)
for t = 1 → 2ec log n do
    R = {1, ..., n}
    for s = 0 → ⌈log n⌉ do
        Compute A^R: a^R_ik = [k ∈ R] · k · a_ik        ▷ [k ∈ R] ist ein Indikator, 1 falls k ∈ R
        Z = A^R B
        for all i, j do
            if W_ij < 0 AND Z_ij is a witness then      ▷ Prüfe: a_{i,Z_ij} = 1 und b_{Z_ij,j} = 1
                W_ij = Z_ij
        R = (1/2)-sample from R
for all i, j do
    if W_ij < 0 then
        Finde Zeugen für i, j mittels Brute Force       ▷ ∀k ∈ R : a_ik = 1 und b_kj = 1
return W
```

6.4 Nachfolger für kürzeste Pfade

Wir nutzen nun die Techniken der Zeugensuche für die Boolesche Matrixmultiplikation, um eine Nachfolgermarix S für das All-Paar-kürzeste-Pfade-Problem (APSP) zu erhalten. Für ein Paar von Knoten (i, j) ist ein Knoten s ein Nachfolger von i auf einem kürzesten Pfad von i nach j genau dann, wenn die Distanz von s nach j genau eins kleiner ist als die Distanz von i nach j, also $d_{sj} = d_{ij} - 1$.

Intuitiv könnte man eine Boolesche Matrix F definieren, wobei $f_{sj} = 1$ ist, wenn $d_{sj} = d_{ij} - 1$. Dann könnte ein Nachfolger für (i, j) als der (i, j)-Eintrag einer Zeugenmatrix für das Boolesche Produkt AF gefunden werden. Das Problem hierbei ist jedoch, dass die Definition von d_{sj} von i abhängt, was bedeuten würde, dass wir alle n Möglichkeiten durchprobieren müssten, was ineffizient wäre.

Das Problem lässt sich vereinfachen, indem nur die Distanzen modulo 3 unterschieden werden. Jeder Pfad einer jeden Länge im Graphen wird mit der Restklasse modulo 3 in den Matrizen $F^{(c)}$, $c = 0, 1, 2$, abgebildet, wobei $f^{(c)}_{sj} = 1$, falls $d_{sj} \equiv_3 (c - 1)$. Das heißt

$$F^{(0)}\text{enthält alle Pfade der Länge } 0, 3, 6, \ldots,$$
$$F^{(1)}\text{enthält alle Pfade der Länge } 1, 4, 7, \ldots,$$
$$F^{(2)}\text{enthält alle Pfade der Länge } 2, 5, 8, \ldots.$$

Der Algorithmus MM-APSP erhält als Eingabe eine $n \times n$ Matrix A und berechnet dazu die Nachfolgermatrix S für kürzeste Pfade. Der Algorithmus MM-APSD hat eine Laufzeit von $O(M(n) \log n)$. In Zeile 1 wird Algorithmus MM-APSD(A) aufgerufen, der für die Adjazenzmatrix A die kürzesten Distanzen für alle (i, j) in $O(M(n) \log(n))$ ermittelt. Der Hauptteil des Algorithmus beinhaltet drei Aufrufe von BMM-Witness, einmal für jedes $c \in \{0, 1, 2\}$. Jeder Aufruf von BMM-Witness hat eine erwartete Laufzeit von $O(M(n) \log^2 n)$. Daher ist die erwartete Gesamtlaufzeit des Algorithmus MM-APSP $O(M(n) \log^2 n) = O(n^{2.3219} \log^2 n)$, wenn Strassen-25 für die Matrixmultiplikation verwendet wird.

Betrachtet man die notwendigen Operationen für Matrixmultiplikation aus Tabelle 2, hat MM-APSP für $n = 1000$ eine erwartete Anzahl von notwendigen Operationen von etwa $10^9 \cdot 99.32$, da $\log^2(10^3) \approx 99.32$, und für $n = 10^6$ eine erwartete Anzahl von notwendigen Operationen von etwa $10^{13} \cdot 397.27$, da $\log^2(10^6) \approx 397.27$.

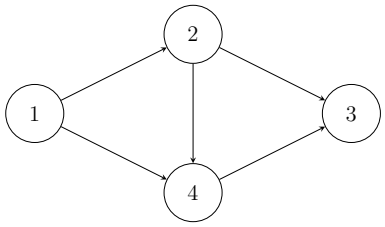
Algorithmus 5 MM-APSP(A)

Eingabe: $\langle A \rangle$ mit $n \times n$ Matrix A
Ausgabe: $\langle S \rangle$ mit $n \times n$ Nachfolgermatrix S

```
D = MM-APSD(A)                                  ▷ Berechne alle kürzesten Distanzen
for c = 0, 1, 2 do
    Definiere F^(c): f_sj^(c) = 1 falls d_sj ≡_3 (c − 1)
    W^(c) = BMM-Witness(A, F^(c))               ▷ Verwende Algorithmus 4
Initialisiere S als n × n Matrix
for all i, j do
    s_ij = w_ij^(d_ij (mod 3))                  ▷ Wähle den Zeugen aus der passenden W^(c) Matrix
return S
```

6.5 Beispiel

Gegeben sei $G = (V, E)$ mit $V = \{1, 2, 3, 4\}$ und $E = \{(1, 2), (1, 4), (2, 3), (2, 4), (4, 3)\}$ und der Adjazenzmatrix A mit $a_{ij} = 1$ genau dann, wenn $(i, j) \in E$.



$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Zunächst erfolgt die Berechnung der kürzesten Distanzen mit MM-APSD(A):

$$D = \begin{pmatrix} 0 & 1 & 2 & 1 \\ \infty & 0 & 1 & 1 \\ \infty & \infty & 0 & \infty \\ \infty & \infty & 1 & 0 \end{pmatrix}$$

Für jedes $c \in \{0, 1, 2\}$ erstellen wir eine Matrix $F^{(c)}$, wobei $f_{sj}^{(c)} = 1$ falls $d_{sj} \equiv_3 (c - 1)$.
$F^{(0)}$ **(c-1 = -1 $\equiv_3$ 2):** $f_{sj}^{(0)} = 1$, falls $d_{sj} \equiv_3 2$

$$F^{(0)} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$F^{(1)}$ **(c-1 = 0):** $f_{sj}^{(1)} = 1$, falls $d_{sj} \equiv_3 0$

$$F^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$F^{(2)}$ **(c-1 = 1):** $f_{sj}^{(2)} = 1$, falls $d_{sj} \equiv_3 1$

$$F^{(2)} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Dann werden die Witness Matrizen ermittelt mit
$W^{(0)} = \textbf{BMM-Witness}(A, F^{(0)})$**:** Berechnung von $A \times F^{(0)}$:

$$W^{(0)} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$W^{(1)} = \textbf{BMM-Witness}(A, F^{(1)})$**:** Berechnung von $A \times F^{(1)}$:

$$W^{(1)} = \begin{pmatrix} 0 & 2 & 0 & 4 \\ 0 & 0 & 3 & 4 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$W^{(2)} = \textbf{BMM-Witness}(A, F^{(2)})$**:** Berechnung von $A \times F^{(2)}$:

$$W^{(2)} = \begin{pmatrix} 0 & 0 & 2 & 2 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Für jede Position (i, j) wird der Zeuge gewählt aus $W^{(d_{ij} \bmod 3)}$ und die Nachfolgermatrix S bestimmt:

$$S = \begin{pmatrix} 0 & 2 & 2 & 4 \\ 0 & 0 & 3 & 4 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

7 Zusammenfassung

In dieser Arbeit wird der Algorithmus von Strassen zur effizienten Multiplikation von Matrizen vorgestellt. Der Strassen-Algorithmus multipliziert zwei $n \times n$ Matrizen A und B unter Verwendung von nur 7 Matrixmultiplikationen anstelle der üblichen 8. Die Laufzeit des Algorithmus wird durch die Rekurrenzgleichung $T(n) = 7 \cdot T\left(\frac{n}{2}\right) + c \cdot n^2$ beschrieben. Der Algorithmus von Strassen hat damit eine Laufzeit von $O(n^{2.8074})$, welche analysiert wird. Der Algorithmus wird in einem Pseudocode vorgestellt. Er unterteilt die Matrizen in 4 Teilmatrizen und berechnet 7 Produkte, die dann zur Berechnung der 4 Blöcke der Ergebnismatrix verwendet werden.

Eine Python-Implementierung des Algorithmus wurde erstellt, um die theoretische Laufzeitanalyse zu überprüfen. In Experimenten werden Matrizen unterschiedlicher Größe (von 4 bis 256) getestet. Die Ergebnisse zeigen, dass für kleinere Matrizen der Standardalgorithmus schneller ist, während für größere Matrizen Strassens Algorithmus überlegen ist. Ein Schwellwert von 32 wird gesetzt, um bei kleinen Matrizen auf die Standardmultiplikation umzuschalten.

Eine Methode zur Reduzierung der Matrixmultiplikationen von 7 auf 5 wird beschrieben, was die Laufzeit auf $O(n^{2.3219})$ senkt. Diese Optimierung nutzt die Diagonale der Matrix B und erfordert vorherige Berechnungen der Produkte. Der Algorithmus Strassen-25 setzt diese Methode um. Das *optimale Teile-und-Herrsche Prinzip* wird vorgestellt und gezeigt, dass es die effizienteste Methode ist, um Probleme zu lösen, wenn die Probleminstanz bei jedem rekursiven Aufruf halbiert wird. Komplexitätsklassen werden eingeführt, bewiesen dass $P = NP$ ist und erwartete Laufzeitergebnisse des Algorithmus von Strassen-25 vorgestellt. Es wird gezeigt, dass schneller als mit Laufzeit $O(n^{2.3219})$ zwei $n \times n$ Matrizen *nicht* multipliziert werden können.

Es wird gezeigt, dass das *Binärsystem optimal für die Signalübertragung* ist. Die Shannon-Hartley-Gleichung wird vorgestellt, um die maximale Datenrate über einen Kommunikationskanal zu beschreiben, wobei die Bedeutung des Signal-Rausch-Verhältnisses hervorgehoben wird.

Der Algorithmus MM-APSP zur Berechnung der kürzesten Wege für alle Knoten in einem Graphen wird behandelt. Das Boolesche Produkt von Matrizen wird verwendet, um die Erreichbarkeit zwischen Knoten zu bestimmen. Ein randomisierter Algorithmus wird vorgestellt, der eine erwartete Laufzeit von $O(M(n)\log(n))$ hat, wobei $M(n)$ die Laufzeit der Matrixmultiplikation ist. Dieser berechnet Zeugen auf dem kürzesten Weg eines jeden Knoten und wird dann von MM-APSP aufgerufen. Es wird gezeigt, dass er Algorithmus MM-APSP damit insgesamt eine erwartete Laufzeit von $O(n^{2.3219}\log^2 n)$, wenn für die Matrixmultiplikation Strassen-25 verwendet wird.

8 Ausblick

Es wurde gezeigt, dass $\mathbf{P} = \mathbf{NP}$ und damit lohnt es sich, auch die Probleme genauer zu betrachten, von denen man bisher vermutete, dass es für sie keinen Algorithmus gibt, der das Problem effizienter, d.h., in polynomieller Laufzeit lösen kann.

Es werden Komplexitätsklassen $\mathbf{L}$, $\mathbf{P_2}$ und $\mathbf{P_3}$ vorgestellt, die jeweils die Menge aller Probleme enthalten, deren Algorithmen nach einem *charakteristischen Lösungsprinzip* arbeiten. Zum Beispiel sind in der Klasse $\mathbf{L}$ alle Probleme enthalten, deren Algorithmen nach dem optimalen Teile-und-Herrsche Prinzip arbeiten. In der Klasse $\mathbf{P_2}$ sind alle Probleme enthalten, deren Algorithmen nach dem Prinzip der dynamischen Programmierung arbeiten und deren Lösung quadratische Laufzeit benötigt. Die Klassen $\mathbf{L}$, $\mathbf{P_2}$ und $\mathbf{P_3}$ sind in $\mathbf{P}$ enthalten aber untereinander enthalten sie sich nicht:

$$\mathbf{L} \subset \mathbf{P}, \quad \mathbf{P_2} \subset \mathbf{P}, \quad \mathbf{P_3} \subset \mathbf{P}, \quad \mathbf{L} \not\subset \mathbf{P_2}, \quad \mathbf{P_2} \not\subset \mathbf{P_3}, \quad \mathbf{L} \not\subset \mathbf{P_3}.$$

In Zukunft macht es Sinn, diesen Gedanken der Komplexitätsklassen konsequent zu nutzen, um weitere Komplexitätsklassen zu definieren. Dann lohnt es sich, Probleme zu betrachten, für die evtl. ein effizienteres Lösungsprinzip infrage kommt. Damit wird der Lösungssuche das Lösungsprinzip für die jeweilige Komplexitätsklasse mitgegeben, was das Finden einer effizienteren Lösung vereinfachen kann.

Der Algorithmus von Strassen-25 für die Matrixmultiplikation von zwei $n \times n$ Matrizen hat eine Laufzeit von $O(n^{2.3219})$. Ruft der Algorithmus MM-APSP für die Matrixmultiplikation Strassen-25 auf, hat MM-APSP eine erwartete Laufzeit von $O(n^{2.3219}\log^2 n)$.

In der Beweisführung, dass es keinen Teile-und-Herrsche Algorithmus zur Multiplikation von $n \times n$ Matrizen gibt, der weniger als 5 Matrixmultiplikationen benötigt, wird die Eigenschaft des Projezierens genutzt. Zwei Vektoren $\boldsymbol{u}$ und $\boldsymbol{v}$ projezieren $\boldsymbol{w}$ genau dann, wenn es Konstanten k_1 und k_2 gibt, so dass

$$k_1\boldsymbol{u} + k_2\boldsymbol{v} = \boldsymbol{w} \neq \mathbf{0}.$$

Es ist im Allgemeinen von Interesse, eine minimale Projektion zu finden: Finde eine minimale Projektion, nimm noch eine Konstante k weg, dann ist es die *größte Einheit*, die aus sich heraus, egal wie, nichts mehr projezieren kann außer $\mathbf{0}$. Die Idee des Algorithmus von Strassen-25 eignet sich zum Finden von größten, nicht projezierbaren Einheiten.

Interessant ist, welche anderen Anwendungsbereiche auch davon profitieren könnten, wenn Strassen-25 für die Matrixmultiplikation verwendet wird.